Nuzaer Omar (nom8m)
Id: 12594726

From Issues to Insights: RAG-based Explanation Generation from Software Engineering Artifacts

The increasing complexity of modern software has created a significant need for "Explainable Systems" to foster user trust and transparency. However, traditional documentation is often incomplete or outdated due to the fast-paced nature of development. This paper proposes a novel solution by leveraging issue-tracking systems (e.g., GitHub, Jira) as dynamic knowledge sources for generating natural language explanations using Retrieval-Augmented Generation (RAG). Instead of relying on a model's internal and potentially hallucinated memory, the authors built a system that actively searches through a project's issue history to find answers.

The authors implement a proof-of-concept Retrieval-Augmented Generation (RAG) pipeline consisting of three stages: Indexing, Retrieval, and Generation. During Indexing, issue data is divided into manageable chunks and stored in a Chroma vector database using semantic embeddings. The Retrieval stage employs advanced RAG techniques, including LLM-based query rewriting and reranking via Cohere, to ensure the most relevant and diverse context is selected while avoiding the lost-in-the-middle effect. Finally, the Generation stage uses locally-deployed, open-weight LLMs, like Ollama to produce explanations grounded strictly in the retrieved data. To test the approach, the authors build a proof-of-concept system using the Mattermost repository, drawing from a large public issue dataset of about 10,000 closed issues so that the content reflects the resolved changes. They used advanced RAG techniques to find the top 15 most relevant pieces of information and their four evaluation criteria being set as accuracy, helpfulness, faithfulness and relevancy. In their system-specific QA experiments, the method performs strongly, showing very high document relevance (94–100%), high helpfulness (over 98%), and faithfulness that often exceeds 90%, with the best models aligning well with documentation derived reference answers. Overall, the results suggest that issue driven RAG can produce explanations that are comparable to documentation-based ones, while staying grounded in real project artifacts and benefiting from the fact that issue trackers evolve alongside the software itself.

A key limitation of the paper is that the proposed advanced retrieval pipeline is evaluated only on the Mattermost repository, and the authors do not provide a concrete demonstration of how LLM based query rewriting behaves on realistic, underspecified user questions. In practice, many user queries are extremely generic (e.g., "Why is the file not generating?"), and such symptoms can arise from numerous unrelated causes like incorrect path configuration, missing parameters, permission issues, environment misconfiguration, or other workflow-specific constraints. Without showing how the rewriting module transforms these vague queries into targeted alternatives, it remains unclear whether rewriting consistently increases retrieval precision (1) or simply expands the search space in a way that introduces redundancy.

A practical improvement would be to add an intermediate step that elicits minimal diagnostic detail before rewriting and retrieval. Once a user submits a question, the system should prompt them to provide the specific error message, or log output. It can also be a multiple query-reply based conversational thread (2). The LLM can then reconstruct a higher quality query by combining the user's intent with these discriminative error cues, enabling more precise semantic search and reducing the need for query expansion.

(1) - Park, Joon, et al. "Emulating Retrieval Augmented Generation via Prompt Engineering for Enhanced Long Context Comprehension in LLMs." *arXiv preprint arXiv:2502.12462*(2025).
(2) - Wang, Xi, et al. "Adaptive retrieval-augmented generation for conversational systems." *Findings of the Association for Computational Linguistics: NAACL 2025*. 2025.